

ÇALIŞMA ŞARTNAMESİ

CC1352R Platformlarında Contiki-NG Telsiz Donanım Yazılımının Güncellemesi

OTA / ELF ANALİZİ / CC1352R

Bu belge, Contiki-NG'de kontrollü telsiz firmware (bellenim) aktarım sistemi için dönem çalışması gereksinimlerini tanımlamaktadır. Çalışma, parçalı firmware dağıtımı, sağlama fonksiyonu ve yeniden iletim stratejisi, alıcı düğümde kalıcı depolama, ELF denetimi ve TI CC1352R Sensortag/Launchpad için donanım uyarlama analizini kapsamaktadır.

OMÜ

BİL 304 - İşletim Sistemleri

Bahar

2025/2026

OTA - Over The Air / Havadan Güncelleme

Bu ödev, Telsiz Duyarga Ağları (Wireless Sensor Networks - WSNs) görevi icra eden gömülü sistem cihazları arasında güncellenmiş bir firmware (bellenim) dosyasının denetimli olarak gönderilmesi ve gönderim sonrası yeni firmware ile başlatılması (reboot) için gereken bir takım araştırma ve geliştirme süreçlerini içermektedir.

1. Geliştirme Süreci

Cooja simülatörü kullanarak Contiki-NG işletim sistemi çalıştıran iki cihaz arasında, OTA (Over-the-Air) prensibiyle firmware aktarım mekanizması geliştirilmesini kapsamaktadır. Çalışmada gönderici düğüm, kendisine önceden verilmiş güncel firmware dosyasını parçalar halinde alıcı düğüme iletmelidir. Alıcı düğüm ise bu parçalı veriyi almalı, tutarlılık doğrulamasından geçirmeli, kendi kalıcı depolama alanına yazmalı ve kaydettiği firmware imajını daha sonra etkinleştirilebilecek bir güncelleme olarak birleştirip saklaması gerekmektedir.

Geliştirme sürecinde, gerçek donanımlardaki bootloader zincirinin birebir gerçekleşmesi beklenmemektedir. Bunun yerine, Cooja ve Contiki-NG' nin elverdiği ölçüde bir kablolu firmware gönderiminin denetimli ve kontrollü iletilmesi beklenmektedir. Dolayısıyla geliştirme sürecinin temel odağı, yeni firmware imajının ağ üzerinden güvenilir biçimde iletilmesi, alıcı düğümde de dosya veya flash benzeri kalıcı bir alana yazılması, imajın bütünlük kontrolünün yapılması yeterlidir.

Verilenler

Geliştirme için gereken şablon verilmiştir. [İndirmek için tıklayınız](#). Sıkıştırılmış olarak verilen klasör açılıp makinenizdeki `contiki-ng/examples/` dizini altına kopyalanmalıdır. Bu klasör altında:

- `udp-server.c` : Gönderici düğüm için başlangıç uygulama kodu
- `udp-server.z1` : Gönderici düğüm için derlenmiş başlangıç uygulaması
- `udp-client.c` : Alıcı düğüm için başlangıç uygulama kodu
- `udp-client.z1` : Alıcı düğüm için derlenmiş başlangıç uygulaması
- `new-firmware.z1` : Kablolu olarak aktarılacak güncel firmware imajı
- `BIL304-OS-Project-1.csc` : Cooja senaryosu ve simülasyon altyapısı

Verilen `rpl-udp` uygulaması, başlangıç bir UDP haberleşme örneğidir. Projede geliştirme sürecinde beklenen, bu iskeleti kullanarak kilobaytlar büyüklüğünde diskte yer kaplayan bir OTA firmware gönderme/alma ve saklama sistemine dönüştürülmesidir.

Gönderilecek `new-firmware.z1` dosyası ham veri değil, ELF biçiminde bir MSP430 sistemlerde koşulabilir bir dosyadır. Dosya boyutu yaklaşık 129760 bayt düzeyindedir. Dolayısıyla bu imaj tek paket halinde taşınmamaktadır. Parçalama, yeniden gönderim ve bütünlük denetimi (checksum) zorunlu olarak ele alınmalıdır.

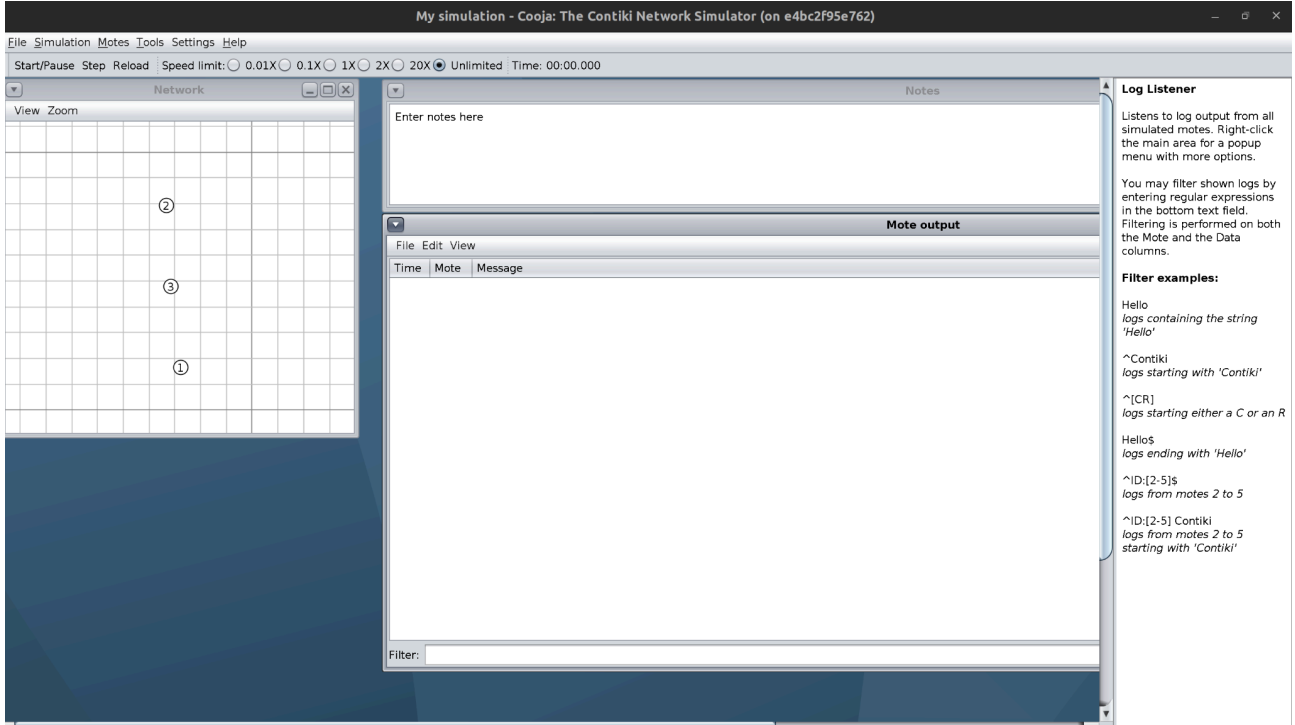
Problem Tanımı

Cooja simülatörü açıldığında üst menüden *File > Open Simulation > Browse* menüsünden `BIL304-OS-Project-1.csc` hazırlanmış simülasyon dosyası seçildiğinde bir gönderici, bir komşu iletilici ve bir de alıcı düğümlü senaryo otomatik olarak açılacaktır. Şekil.1' de açılan simülasyonun ekran görüntüsü verilmiştir. Burada 2 ve 3 ID değerli düğümlere `udp-client.z1`; 1 ID değerli düğüme de `udp-server.z1` yüklenmiştir. Varsayılan hali ile simülasyon çalıştırıldığında, 2 ID' li düğüm tarafından her adımda artan bir tamsayı değeri `tx_count`, Merhaba mesajı ile birleştirilip gönderilmektedir (Örn: "Merhaba 232"). Burada 3 numaralı düğüm, 2 numaralı düğüm tarafından 1 numaralı düğüme gönderilmek istenen tüm mesajları ileten aracı komşu görevi görmektedir.

Soru: 2 ve 3 numaralı düğüme aynı firmware yüklenmesine rağmen gönderim işlemi neden sadece 2 numaralı düğüm tarafından yapılmaktadır, 3 numaralı düğüm de mesaj göndermesi gerekmez mi?

Cevap: Her 2 cihaza da aynı kaynak kodunun derlenmiş hali yüklenmesine rağmen kod içerisinde `node-id` kütüphanesi ve `if` blokları kullanılarak; sistem, içerisine yüklendiği cihazın ID değerini donanıma sormakta,

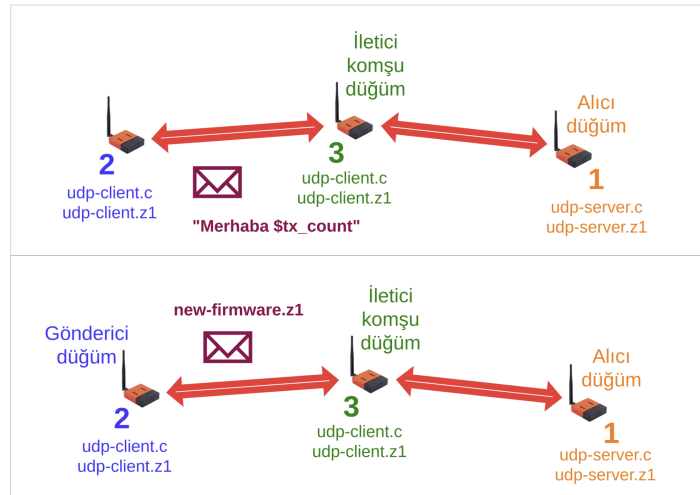
eğer bu değer “2” ise bir takım kod bloklarını, değilse bir başka kod bloğunu koşturmaktadır. `node-id` ve `if` blokları sayesinde her cihaz için farklı firmware hazırlama zahmetinden kurtulmuş olunmaktadır. (Bkz. `udp-client.c` 2 ve 3 numaralı düğümlere yüklenmesine rağmen 70. satırdaki kod bloğu sadece 2 numaralı düğüm içine yüklendiğinde çalışacaktır.)



Şekil.1 Proje kapsamında verilen hazır senaryo `BIL304-OS-Project-1.csc`’nin açılması

Gönderici düğüm (ID:2), `new-firmware.z1` dosyasını bloklara ayırarak alıcı düğüme göndermelidir. Hali hazırda yapılan “Merhaba ...” dizge (string) mesajı gönderimi, bayt dizisi göndericisine dönüştürülmelidir. Alıcı düğüm (ID:1), aldığı bayt bloklarını hatalı bit kontrolü yapıp, doğru sırayla birleştirerek kalıcı depolama alanına kaydetmelidir. Ardından “Yüklenmeye hazır yeni firmware alımı tamamlandı.” mesajı ekrana yazdırılmalıdır. 3 numaralı komşu düğüm için `udp-client.c` içerisinde herhangi bir geliştirmeye gerek yoktur. Komşu rolünü icra edebilecek durumdadır.

Kişisel bilgisayarlardaki “disk” ifadesi, gömülü sistemlerde kalıcı saklama alanı olarak yorumlanmalıdır. Gömülü sistemlerde kalıcı saklama alanı için Flash, EEPROM vb. adlandırmalar da kullanılmaktadır. Contiki-NG işletim sistemi tarafından sunulan dosya sistemi Coffee’dir. (Bkz. [dokümantasyon](#), dosya sistemi firmware örnekleri `contiki-ng/examples/storage/cfs-coffee`, `contiki-ng/storage/eeprom-test`) İsteyenler, başka depolama yapıları, CFS arayüzü, harici flash benzetimi veya uygun gördükleri başka bir kalıcı saklama yaklaşımını da kullanabilirler.



Şekil.2 Dizge (string) gönderen taslağın bayt dizisi göndericiye dönüştürülmesi (şablon üstte, istenen altta)

Geliştirilecek çözüm aşağıdaki temel özellikleri sağlamalıdır:

1. Gönderici düğüm, `new-firmware.z1` isimli imajı belleğe almalı veya uygun biçimde okumalıdır.
2. Firmware imajı, sabit boyutlu paketlere veya bloklara bölünmelidir.
3. Her blok için en azından blok numarası, veri uzunluğu ve gerekli kontrol bilgisi taşınmalıdır.
4. Alıcı düğüm, gelen blokları toplamalı ve kalıcı depolama alanına yazmalıdır.
5. Alıcı düğüm, varsa eksik veya bozuk blokları tespit edebilmelidir.
6. Hatalı iletim gerçekleşmişse, tespit sonrası ilgili bloğun yeniden gönderimi tasarlanmalıdır.
7. Aktarım sonunda da tüm imaj için bir bütünlük doğrulaması yapılmalıdır.
8. Alıcı düğüm, kendi diskine birleştirilmiş imajı kaydetmelidir.
9. Kayıt sonrası güncellenmenin başarıyla alındığı ve saklandığı gösterilmelidir.
10. Cooja aracında alıcı düğüm tarafından, diskine sakladığı bu makine kodunun boot edilecek başlangıç adresi olarak kaydedilemeyeceği ve reboot edilemeyeceği için ödevin 1. aşamasında reboot beklenmemektedir.

Verilen maddeler doğrultusunda kodlanacak iş parçacıkları şu şekilde verilebilir:

1. Parçalama: Firmware imajının uygun büyüklükte bloklara ayrılması,
2. Sıralama: Her blok için blok numarası veya offset bilgisinin taşınması,
3. Parça doğrulama: Her blok için checksum, CRC veya benzeri bir doğrulama yapılması,
4. Tüm-imaj doğrulama: Aktarım sonunda tüm dosya için hash, CRC32 veya benzeri bir bütünlük mekanizması çalıştırılması,
5. Yeniden gönderim: Bozuk, kayıp veya zaman aşımına uğrayan blokların tekrar istenebilmesi,
6. Tekrarlı gönderim veya pencereleme: Stop-and-wait, sliding window, seçmeli tekrar gönderim veya benzeri bir stratejinin gerçekleştirilmesi,
7. Durum yönetimi: Alıcı düğüm, hangi blokların geldiğini ve hangilerinin eksik olduğunu izleyebilmesi.

İş parçacıkları doğrultusunda tasarlanan ve gerçekleştirilen güvenilir aktarım stratejisi raporda izah edilmelidir.

GÜNCELLEME - ÖRNEK KOD PAYLAŞIMI - 18.05.2026

İmaj dosyasını hex formatında `.h` dosyasına kaydedip gönderebilirsiniz. [Örnek firmware-i](#) contiki-ng/examples klasörünüze çıkarınız. (firmware_data.h dosyasını inceleyiniz)

2. Araştırma Süreci

Projenin birinci bölümü olan geliştirme süreci, cihazlara yüklenecek yeni firmware' in güvenli iletilmesi ve hedef cihazın diskine saklanmasını ihtiva etmekte idi.

İkinci bölüm olan araştırma sürecinde ise yüklenecek firmware dosyasının geliştirici araç zinciri (toolchain) kullanılarak analiz edilmesini ve Texas Instruments CC1352R Sensortag veya Launchpad SoC (System-on-Chip) cihazlarda OTA' nın gerçekleştirilmesi için gereken yükleyici (loader) implementasyonu ve raporlanmasını kapsamaktadır.

1. ELF Analizi

ELF analiz bölümü, farklı görevler için geliştirilmiş firmware imajlarının yapısal olarak icra edilmesinin yanında, bellek ve disk uzayına oturma stratejilerinin (linker analizi) anlaşılması ve izah edilmesini kapsamaktadır. Öğrenciler farklı firmware dosyalarını analiz etmelidir.

Analizde en az aşağıdaki başlıklar yer almalıdır:

1. Dosyanın ELF sınıfı, mimarisi ve giriş adresi
2. ``.text``, ``.data``, ``.bss``, ``.rodata``, ``.vectors`` gibi temel bölümlerin varlığı ve ifade ettiği anlam

3. Kod ve veri boyutları (Bunların ne anlama geldiği)
4. Sembol tablosu ve anlamlı semboller (Bunların ne anlama geldiği)
5. Kesme vektörleri veya başlangıç adresi ile ilişkili bilgiler (Bunların ne anlama geldiği)
6. Dosyanın neden "ham binary" değil de "ELF executable" olarak değerlendirildiği

Öğrenciler kurulum sonrası gelen araçlardan uygun olanlarını kullanmalıdır. Bu araç zincirleri msp430 c derleyicisinin (msp430-gcc) de bulunduğu bin/ dizini altındadır.

```
- `file x-firmware.z1`  
- `msp430-readelf -h x-firmware.z1`  
- `msp430-readelf -S x-firmware.z1`  
- `msp430-readelf -l x-firmware.z1`  
- `msp430-readelf -s x-firmware.z1`  
- `msp430-objdump -x x-firmware.z1`  
- `msp430-objdump -h x-firmware.z1`  
- `msp430-objdump -d x-firmware.z1`  
- `msp430-nm -n x-firmware.z1`  
- `msp430-size x-firmware.z1`  
- `msp430-strings x-firmware.z1`
```

Ve dizinde bulunan diğer araç zincirleri (Örneğin: msp430-addr2line, msp430-ld, msp430-strip, msp430mcu-config ...)

Rapor bölümünde yalnızca komut çıktısını kopyalamak yeterli değildir. Araç zincirinin kullanım amacı ve çıktılarının anlamı yorumlanmalı ve firmware imajının rolüne ilişkin sonuçlar açıklanmalıdır. Araç zinciri ile üretilen disk ve bellek uzayına ait sonuçlar, CC1352R SoC Teknik Dokümantasyonunda verilen disk ve bellek uzaylarına uygun olarak görselleştirilmelidir. Örneğin X firmware, SoC' nin disk ve bellekteki hangi bölgelerinde, ne kadar yer kaplayacaktır ? vb. sorular... (SoC' nin Teknik Dokümantasyonları paylaşılmıştır.)

! Talep edilirse analiz için farklı icraatları olan firmware dosyaları paylaşılacaktır.

Paylaşılan SoC Dokümanları

- `TI-Docs/CC1352R SimpleLink High-Performance Multi-Band Wireless MCU datasheet (Rev. I).pdf`
- `TI-Docs/CC13x2, CC26x2 SimpleLink Wireless MCU Technical Reference Manual (Rev. G).pdf`
- `TI-Docs/CC1352R SimpleLink™ Wireless MCU Device Revision E Silicon Errata (Rev. C).pdf`

3. CC1352R Gerçekleme ve Donanım Uyarılama Görevi

Bu görevde OTA işleminin 3.bileşeni olan gerçek cihaz üzerinde, cihazın diskinde data olarak oturan yeni firmware ile reboot edilmesi problemini kapsamaktadır. Bölüm envanterinde bulunan `TI CC1352R LaunchPad ve Sesnortag` cihazlarında geliştirme yapılabilmesi için cihaza uygun araç zincirleri yüklenmelidir. Ödevin ilk 2 iş parçacığı msp430 çekirdek mimarisi üzerinde iken; 3. iş parçacığı ise CC1352R SoC çekirdeğini oluşturan ARM mimarisi üzerindedir. Dolayısıyla Contiki-NG işletim sistemini ARM platformlarında koşan programların üretilmesi için ARM araç zincirleri [linkten yüklenmelidir](#). Bu iş parçacığı ile uğraşmak isteyen takımlar, donanımın ortak kullanımı için dersin Araştırma Görevlisi ile iletişime geçmelidir.

Contiki-NG platform bilgilendirme [sayfasını okuyunuz](#). Bu bölümde de, çalışma dizinindeki `TI-Docs` klasörü altında bulunan aşağıdaki teknik dokümanlara ihtiyaç bulunmaktadır.

- `TI-Docs/CC1352R SimpleLink High-Performance Multi-Band Wireless MCU datasheet (Rev. I).pdf`
- `TI-Docs/CC13x2, CC26x2 SimpleLink Wireless MCU Technical Reference Manual (Rev. G).pdf`
- `TI-Docs/CC1352R SimpleLink™ Wireless MCU Device Revision E Silicon Errata (Rev. C).pdf`

Bu iş parçacığı tek cihaz üzerinde çalışacak şekilde tasarlanmıştır. Reboot edilecek yeni firmware için gönderim işlemi yapılmayacaktır. Tek cihaza eski firmware flash-lanırken, yeni firmware de cihazın diskinde kaydedilecek data olarak yüklenmelidir. TI cihazlara firmware yükleyici online UNIFLASH aracı [için tıklayınız](#).

CC1352R Launchpad üzerinde yeni firmware ile reboot için örnek araştırma konuları

1. CC1352R'nin Flash, SRAM, cache/GPRAM ve ROM kapasitesi
2. Uygulama kodunun hangi bellekte tutulduğu

3. Flash aşamasında yeni imajın hangi fiziksel veya mantıksal bölgeye yerleştirileceği
4. Güncelleme için tek imaj, çift imaj veya geçici staging alanı gerekip gerekmediği
5. Boot sürecinde ROM bootloader ve CCFG alanının rolü
6. Reset sonrası hangi koşullarda hangi imajın seçileceği
7. Flash erase/write işlemlerinin neden dikkatli planlanması gerektiği
8. RAM kısıtları nedeniyle aktarım tamponlarının nasıl boyutlandırılması gerektiği

3. İş parçacığının çalışan gösteriminin yanında dokümantasyonunda beklenen çıktılar:

1. Flash ve RAM kullanımını gösteren bir bellek yerleşim tablosu
2. Boot zincirini gösteren kısa bir akış diyagramı
3. CCFG alanının neden kritik olduğunu açıklayan kısa teknik not
4. Diske 2.firmware kayıt için önerilen yerleşim stratejisi
5. Eğer tam firmware değiştirme yapılacaksa, bunun neden bootloader gerektirdiğinin açıklanması



Şekil.3 TI CC1352R ailesi 2 ürünü Sensortag (sol üstte) ve Launchpad cihazı (sağ altta)

Peşinden gidilecek sorular

- Uygulamanın çalışan ana imajı nereye yerleşecek?
- Diske kaydedilecek yeni imaj hangi alana yazılacak?
- Aynı anda iki tam imaj saklanabiliyor mu?
- Sadece staging alanı varsa aktivasyon nasıl yapılacak?
- Flash sektör silme ve yazma işlemleri mevcut çalışan imajı nasıl etkileyebilir?

Puanlama

- 1- Geliştirme İş Parçacığı, 0-50 puan
- 2- Araştırma İş Parçacığı, 0-30 puan
- 3- CC1352R Gerçekleme ve Donanım Uyarılama İş Parçacığı, 0-100 puan

[Simülasyon şablonu](#)' nu [contiki-ng/examples/](#) dizi altına yerleştiriniz.

TI dokümantasyonlar

Ödev Grupları Girişi

Gruplardaki üye sayısı en az 1, en fazla 4 olmalıdır.

Ödev grubu giriş son tarih **3 Mayıs !**

Başarılar

Doç. Dr. Sercan DEMİRCİ

Arş. Gör. İsmail Hakkı TURAN